

POLITECHNIKA RZESZOWSKA
KATEDRA INFORMATYKI I AUTOMATYKI
DR INŻ. MATEUSZ POMIANEK

ĆWICZENIE LABORATORYJNE REACT NATIVE 1

TypeScript · Komponenty · Nawigacja · REST API

Programowanie Aplikacji Mobilnych - moduł Flutter

Projekt: WeatherApp - aplikacja pogodowa z Open-Meteo API

Wymagania wstępne: znajomość podstaw JavaScript/TypeScript, zainstalowany Node.js i Android Studio

Spis treści

1. Czym jest React Native i jak działa?	1
2. Instalacja i konfiguracja środowiska	2
3. Struktura projektu React Native	4
4. TypeScript - podstawy języka w kontekście RN	5
5. Komponenty - fundamenty UI w React Native	6
6. Stan i efekty - useState, useEffect i Context API	8
7. Nawigacja - React Navigation i Expo Router	10
8. Pobieranie danych z REST API	12
9. Stylowanie - StyleSheet i nowoczesne podejścia	14
10. Projekt WeatherApp - architektura i struktura	15

1. Czym jest React Native i jak działa?

React Native to otwartoźródłowy framework firmy Meta (dawniej Facebook) do tworzenia aplikacji mobilnych przy użyciu JavaScript i TypeScript. W odróżnieniu od zwykłych aplikacji webowych opakowanych w WebView (np. Ionic), React Native kompiluje komponenty do PRAWDZIWYCH natywnych widżetów systemowych - przycisk w RN to prawdziwy Android Button, nie jego kopia rysowana na płótnie.

W tym ćwiczeniu skupimy się na budowaniu aplikacji pogodowej WeatherApp, która pobiera dane z otwartego API Open-Meteo i wyświetla je w natywnym interfejsie.

Analogia: tłumacz symultaniczny React Native działa jak tłumacz symultaniczny podczas międzynarodowej konferencji. Ty mówisz po angielsku (JavaScript), a tłumacz natychmiast przekazuje treść po polsku systemowi Android i po japońsku systemowi iOS. Słuchacze słyszą swój ojczysty język - widzą prawdziwe natywne komponenty, nie imitacje.

1.1. Dwa mosty w architekturze React Native

React Native przeszedł gruntowną przebudowę architektury. Stara architektura (Bridge) i nowa (JSI + Fabric) różnią się fundamentalnie sposobem komunikacji między kodem JavaScript, a kodem natywnym.

Cecha	Stara architektura (Bridge)	Nowa architektura (JSI + Fabric)
Komunikacja JS ↔ Native	Asynchroniczny, serializowany most JSON. Każde wywołanie to marshal/unmarshal danych.	Synchroniczne wywołanie przez JSI (JavaScript Interface). Brak serializacji.
Renderowanie	Shadow Tree w JS, potem aktualizacja UI na watku natywnym.	Fabric renderer: synchroniczne, współdzielone drzewo. Brak 'skoków' UI.
Wydajność	Bottle-neck: most JSON dla gestów i animacji.	Zbliżona do natywnej; animacje na 60/120 fps bez lagów.
Status (RN 0.76+)	Wyłączana domyślnie; dostępna jako opcja kompatybilności.	Domyślna od RN 0.76. Wymagana dla nowych projektów.

1.2. React Native vs inne podejścia crossplatform

Podejście / Przykład	Jak działa / Wady i zalety
React Native (JS/TS)	Most do natywnych widżetów. Prawdziwy UI platformy. Ogromna społeczność frontendowa. Język JS/TS znany web-deweloperom.
Flutter (Dart)	Własny silnik renderujący (Skia/Impeller). Identyczny wygląd na iOS i Android. Szybszy rendering, ale wyglądem nie-platformowe.
Expo (nad RN)	Warstwa nad React Native z gotowymi API (kamera, notyfikacje). Usuwa potrzebę Android Studio/Xcode dla prostych aplikacji. Rekomendowany na start.
Ionic / Capacitor	Aplikacja webowa w WebView. Używa HTML/CSS. Najłatwiejsze przejście z web, ale niejnatywny wygląd i niższa wydajność.
Kotlin Multiplatform	Współdzieli logikę, UI osobny dla każdej platformy. Najlepsza integracja z natywnym systemem. Wymaga znajomości Kotlin i Swift.

Dlaczego JavaScript/TypeScript? Miliony web-deweloperów znają JS. Meta chciała by mogli tworzyć aplikacje mobilne bez nauki nowego języka. TypeScript (nadzbiór JS z typami) rozwiązuje główną słabość JS: brak typowania statycznego. W 2024 roku praktycznie każdy nowy projekt RN używa TypeScript.

2. Instalacja i konfiguracja środowiska

React Native oferuje dwa sposoby pracy: Expo Go (szybki start bez Android Studio) i React Native CLI (pełna kontrola, wymagany Android Studio). W tym ćwiczeniu użyjemy React Native CLI z Expo jako narzędziem zarządzającym.

2.1. Wymagania wstępne

Wymaganie	Wersja minimalna / uwagi
Node.js	v18 LTS lub nowszy. Sprawdź: <code>node --version</code> . Użyj <code>nvm</code> do zarządzania wersjami.
npm lub Yarn	npm 9+ (z Node.js) lub Yarn 1.22+. W ćwiczeniu użyjemy npm.

Android Studio	Giraffe (2022.3.1) lub nowszy. Wymagany do Android SDK i emulatora AVD.
JDK	JDK 17 (Temurin/OpenJDK). Android Studio instaluje go automatycznie.
Android SDK	API level 33+ (Android 13). Dostępny przez Android Studio SDK Manager.
Git	2.x - do klonowania i zarządzania wersjami projektów.

2.2. Tworzenie projektu z Expo

Expo to framework i platforma dla React Native, która znacznie upraszcza początek pracy. Nie eliminuje Android Studio, ale zarządza dużą częścią konfiguracji natywnej automatycznie.

```
# Instalacja Expo CLI globalnie (jednorazowo):
npm install -g expo-cli

# Tworzenie nowego projektu z szablonem TypeScript:
npx create-expo-app@latest WeatherApp --template blank-typescript

# Przejdź do katalogu i uruchom:
cd WeatherApp
npx expo start

# Skroty klawiszowe w terminalu podczas expo start:
# a = Uruchom na Android emulatorze (wymagany AVD)
# i = Uruchom na iOS symulatorze (tylko macOS + Xcode)
# w = Uruchom w przeglądarce (wersja webowa)
# r = Reload (przeładuj JS bundle)
# d = Otwórz menu deweloperskie w emulatorze
```

Rozróżnienie: Expo Go vs Development Build: Expo Go to aplikacja na telefon/emulator z preinstalowanymi bibliotekami. Wystarczy do większości ćwiczeń. Development Build to pełna wersja z Twoimi natywnymi modułami - potrzebna gdy dołączyć biblioteki z kodem natywnym (np. react-native-camera).

2.3. Konfiguracja Android Studio i AVD

```
# Kroki konfiguracji Android Studio dla React Native:
1. Zainstaluj Android Studio (installer z developer.android.com/studio)
2. Podczas instalacji zaznacz: Android SDK, Android SDK Platform, Android Virtual Device
3. W SDK Manager (Tools > SDK Manager) zainstaluj:
   - Android SDK Platform 34 (Android 14)
   - Android SDK Build-Tools 34.0.0
   - Android Emulator

# Zmienne środowiskowe (dodaj do ~/.bashrc lub ~/.zshrc):
export ANDROID_HOME=$HOME/Android/Sdk # Linux
export ANDROID_HOME=$HOME/Library/Android/sdk # macOS
export PATH=$PATH:$ANDROID_HOME/emulator
export PATH=$PATH:$ANDROID_HOME/platform-tools

# Weryfikacja:
adb devices # Powinna wylistować emulator lub telefon
```

Fast Refresh: Fast Refresh w RN to odpowiednik Hot Reload we Flutterze: zmiana w kodzie JS pojawia się w aplikacji w ciągu sekund bez utraty stanu. Działa automatycznie. Dla zmian w kodzie natywnym (java/kotlin) wymagany jest pełny restart (r w terminalu lub Reload w menu deweloperskim).

3. Struktura projektu React Native

Projekt stworzony przez create-expo-app ma z góry określona strukturę katalogów. Zrozumienie roli każdego pliku jest kluczowe - pomylenie ich to jeden z pierwszych błędów początkujących.

3.1. Mapa katalogów

WeatherApp/	
├─ app/	← GŁÓWNY KATALOG EKRANÓW (Expo Router)
│ └─ (tabs)/	← Grupowanie tras w zakładkach
│ └─ index.tsx	← Ekran główny (zakładka Home)
│ └─ _layout.tsx	← Konfiguracja zakładek
│ └─ detail/	
│ └─ [city].tsx	← Ekran szczegółów (parametr dynamiczny!)
│ └─ _layout.tsx	← Korzeń nawigacji (Stack/Tabs)
├─ components/	← Współdzielone komponenty UI
│ └─ WeatherCard.tsx	
│ └─ ForecastTile.tsx	
├─ hooks/	← Własne React Hooks
│ └─ useWeather.ts	
│ └─ useFavorites.ts	
├─ services/	← Logika API (fetch, axios)
│ └─ weatherService.ts	
├─ types/	← Definicje typów TypeScript
│ └─ weather.ts	
├─ constants/	← Stałe (kolory, URL API, klucze)
│ └─ config.ts	
├─ assets/	← Obrazy, fonty, pliki statyczne
├─ app.json	← Konfiguracja Expo (nazwa, ikona, splash)
├─ tsconfig.json	← Konfiguracja TypeScript
├─ package.json	← Zależności (odpowiednik pubspec.yaml)
└─ babel.config.js	← Konfiguracja transpilatora Babel

3.2. Plik package.json - manifest projektu

Klucz w package.json	Wyjaśnienie
name	Nazwa projektu (małe litery, myślniki). Identyfikator npm.
version	Wersja semantyczna: major.minor.patch. Wyświetlana w ustawieniach aplikacji.
main	Punkt wejścia: 'expo-router/entry' dla Expo Router. Expo wczytuje ten plik jako korzeń.
dependencies	Zależności RUNTIME - pakiety używane w działających aplikacjach (react, react-native, expo).
devDependencies	Zależności DEWELOPERSKIE - tylko podczas kompilacji i testów (typescript, @types/*, jest).
scripts	Skróty poleceń: 'start', 'android', 'ios', 'web', 'test'. Uruchamiane przez: npm run <nazwa>.
expo.sdk	Wersja Expo SDK (w app.json). Definiuje dostępne natywne API.

4. TypeScript - podstawy języka w kontekście RN

TypeScript to nadzbiór JavaScript z opcjonalnym typowaniem statycznym. W React Native jest de facto standardem od 2022 roku. Jeśli znasz JavaScript, TypeScript będzie znajomy - różnice są głównie addytywne (dodajesz typy, nie przepisujesz logiki).

4.1. Typy i interfejsy

Kod TypeScript	Wyjasnienie
<pre>// Podstawowe typy const city: string = 'Warszawa';</pre>	TypeScript dodaje typy do zmiennych; JS ich nie ma Explicit type annotation. Kompilator pilnuje że city jest zawsze stringiem.
<pre>const temp: number = 22.5; const isRaining: boolean = false;</pre>	number obejmuje int i float (jak Dart). Brak osobnych typow. boolean (male litery) - nie Boolean jak w Javie.
<pre>let maybeCity: string null = null;</pre>	Union type: string lub null. Odpowiednik String? w Dart i Kotlin.
<pre>// Interfejs - ksztalt obiekta interface WeatherData { city: string; temperature: number; humidity: number; icon?: string; }</pre>	Interfejsy definiują strukturę obiektów (jak data class) interface = deklaracja kształtu. Brak implementacji. Pole wymagane - musi być przy tworzeniu obiektu. Odpowiednik: val temperature: Double w Kotlin. Pole wymagane. ? = opcjonalne. Może być undefined (brak pola) lub string.

4.2. Typy dla komponentów React Native

Kod TypeScript	Wyjasnienie
<pre>// Props komponentu - zawsze interfejs interface WeatherCardProps { weather: WeatherData; onPress?: () => void; style?: StyleProp<ViewStyle>; }</pre>	Props = właściwości przekazywane do komponentu (jak parametry konstruktora Widgetu Flutter) Konwencja: NazwaKomponentuProps Wymagany prop - brak = błąd TypeScript Opcjonalny callback. () => void = funkcja bez argumentów i wartości zwracanej. Jak VoidCallback w Dart.
<pre>style?: StyleProp<ViewStyle>; }</pre>	Prop stylu - pozwala rodzicowi nadpisać styl. StyleProp<ViewStyle> to typ wbudowany w RN.
<pre>// Typy specyficzne dla RN: import { ViewStyle, TextStyle } from 'react- native';</pre>	Importowane z react-native lub @types/react-native ViewStyle = styl kontenera, TextStyle = styl tekstu. Kompilator pilnuje poprawnych propów CSS-like.

4.3. Funkcje, async/await i generyki

```
// Funkcja z typami - arrow function (najczęstszy styl w RN)
```

```
const formatTemp = (celsius: number): string => {
  return `${celsius.toFixed(1)}°C`; // Template literals: ${wyrazienie}
};

// Async/await - identyczny wzorzec jak w Dart
const fetchWeather = async (city: string): Promise<WeatherData> => {
  const response = await fetch(`${API_URL}?city=${city}`);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json() as WeatherData; // type assertion: 'ufaj mi, to
WeatherData'
};

// Generyki - funkcja wielokrotnego uzytku z typem parametrycznym
const getFirst = <T>(items: T[]): T | undefined => items[0];
const firstCity = getFirst<string>(['Warszawa', 'Krakow']); // string | undefined

// Typ unii z discriminant - elegancka obsługa stanu asynchronicznego
type AsyncState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string };

// Uzycie - TypeScript automatycznie zaweza typ w switch/if:
function render(state: AsyncState<WeatherData>) {
  if (state.status === 'success') {
    return state.data.city; // TypeScript wie ze state.data istnieje!
  }
}
```

5. Komponenty - fundamenty UI w React Native

Jesli Flutter opiera sie na klasach Widget, to React Native opiera sie na funkcjach komponentach. Komponent to funkcja, która przyjmuje props i zwraca JSX - specjalny syntax laczaczy JavaScript z opisem UI. Wszystko w React Native jest komponentem: tekst, przycisk, kontener, cały ekran.

Analogia: klocki LEGO z przepisem Komponent w React Native to jak klocek LEGO z instrukcja: wiesz co dostaniesz (props determinuja wyglad), mozesz go laczyć z innymi klockami i uzywac wielokrotnie. Kazdy klocek zna tylko swój wnetrze - nie wie co jest na zewnatrz. Aplikacja to budowla z klocków.

5.1. Podstawowe komponenty RN - cheat sheet

Komponent RN	Odpowiednik w Androidzie / Compose	Opis i uwagi
View	FrameLayout / Box / Column / Row	Kontener ogolnego przeznaczenia. Nie ma własnego wygladu. Odpowiednik <div> w HTML.
Text	TextView / Text (Compose)	JEDYNY sposob wyswietlania tekstu. Kazdy tekst MUSI być w <Text>.
Image	ImageView / Image (Compose)	Obrazy lokalne (require()) i zdalne (uri:). Zawsze ustawia rozmiar przez style.
TextInput	EditText / TextField (Compose)	Pole tekstowe. Kontrolowane przez useState. Prop value + onChangeText.

ScrollView	ScrollView / LazyColumn (bez lazy)	Przewijana lista. Renderuje WSZYSTKIE dzieci od razu. Użyj FlatList dla długich list.
FlatList	RecyclerView / LazyColumn	Leniwa lista - renderuje tylko widoczne elementy. Zawsze użyj dla długich list.
TouchableOpacity	Button / clickable Modifier	Element klikalny ze zmniejszaniem nieprzezroczystości. Użyj Pressable dla nowych projektów.
Pressable	Modifier.clickable / Button	Nowszy, elastyczny element klikalny. Ma stan pressed do stylowania.
SafeAreaView	WindowInsets / systemBarsPadding	Chroni treść przed notchem, wyrezkami i systemowym paskiem nawigacji.
ActivityIndicator	CircularProgressIndicator	Spinner ładowania. Odpowiednik Flutterowego CircularProgressIndicator.
Modal	AlertDialog / Dialog	Pływające okno modalne z animacją.

5.2. Komponent funkcyjny - WeatherCard

```
// components/WeatherCard.tsx
import React from 'react';
import { View, Text, StyleSheet, Pressable } from 'react-native';
import { WeatherData } from '../types/weather'; // nasz interfejs

// Definicja Props - ZAWSZE interfejs dla komponentów
interface WeatherCardProps {
  weather: WeatherData; // wymagany prop
  onPress?: () => void; // opcjonalny callback
}

// Komponent funkcyjny - arrow function z destrukcją props
export const WeatherCard: React.FC<WeatherCardProps> = ({ weather, onPress }) => {
  // JSX - JavaScript + XML-like syntax do opisu UI
  return (
    <Pressable // klikalny kontener
      onPress={onPress} // {wyrzucenie JS} w JSX = klamry
      style={{({ pressed }) => [ // styl zależy od stanu pressed
        styles.card,
        pressed && styles.pressed // && = warunkowy styl
      ]}}
    >
      <Text style={styles.city}>{weather.city}</Text>
      <Text style={styles.temp}>
        {`${weather.temperature.toFixed(1)}°C`} {/* template literal */}
      </Text>
      <Text style={styles.humidity}>
        Wilgotność: {weather.humidity}%
      </Text>
    </Pressable>
  );
};

// StyleSheet.create - optymalizuje style (przetwarza raz, nie przy każdym renderze)
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 16,
    padding: 16,
    marginVertical: 8,
    elevation: 4, // Cień Android
    shadowColor: '#000', // Cień iOS
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
  },
});
```



```

    shadowRadius: 8,
  },
  pressed: { opacity: 0.85 }, // Efekt naciśnięcia
  city: { fontSize: 22, fontWeight: '700', color: '#1a1a2e' },
  temp: { fontSize: 48, fontWeight: '300', color: '#0164FF' },
  humidity: { fontSize: 14, color: '#666' },
});

```

JSX a Dart widgets: W Dart piszesz: `Text('Warszawa', style: TextStyle(fontSize: 22))`. W React Native piszesz: `<Text style={{fontSize: 22}}>Warszawa</Text>`. Różnice są składniowe, ale koncepcja identyczna - tworzysz hierarchie elementów UI z właściwościami. JSX kompiluje się do `React.createElement('Text', {style:...}, 'Warszawa')` - wywołania JS.

5.3. FlatList - wydajna lista

```

// Komponent z FlatList - odpowiednik ListView.builder w Dart
import { FlatList, ListRenderItem } from 'react-native';

// Typ dla renderItem - TypeScript pomaga uniknąć błędów
const renderWeatherCard: ListRenderItem<WeatherData> = ({ item, index }) => (
  <WeatherCard
    weather={item}
    onPress={() => router.push(`/detail/${item.city}`)}
  />
);

<FlatList<WeatherData>
  data={weatherList} // tablica danych do wyświetlenia
  renderItem={renderWeatherCard} // funkcja tworząca element dla każdego item
  keyExtractor={(item) => item.city} // unikalny klucz (jak key w Compose)
  // Stany specjalne:
  ListEmptyComponent={<Text>Brak danych</Text>}
  ListHeaderComponent={<SearchBar />} // Nagłówek listy (przewija się razem)
  // Odświeżanie (Pull-to-Refresh):
  refreshing={isLoading}
  onRefresh={loadData}
  // Optymalizacje:
  initialNumToRender={10} // Ile elementów renderować przy starcie
  windowSize={5} // Ile 'okien' ekranu w pamięci
/>

```

6. Stan i efekty - useState, useEffect i Context API

React Native używa systemu hooków do zarządzania stanem i efektami ubocznymi. Hooki to specjalne funkcje rozpoczynające się od 'use', które 'podpinają' komponent do mechanizmów Reacta. Są odpowiednikami `setState()` + `StatefulWidget` z Fluttera, ale elegantsze.

6.1. useState - lokalny stan komponentu

Kod React Native	Wyjaśnienie
<pre>import { useState } from 'react'; const [count, setCount] = useState(0);</pre>	Importuj hook <code>useState</code> z pakietu <code>react</code>
<pre>const [city, setCity] = useState<string>('');</pre>	Destrukturyzacja: <code>[wartosc, setter] = useState(wartoscPocatkowa)</code> . Jak para <code>state + setState()</code> w Dart.
<pre>const [weather, setWeather] =</pre>	Jawny typ generyczny. Bez niego TS wnioskuje typ z wartości początkowej.
	Stan nullable - przed załadowaniem danych jest null.


```

    useState<WeatherData |
    null>(null);

    setCity('Gdansk');

    setCount(prev => prev + 1);

```

WeatherData | null = typ unii. Odpowiednik WeatherData? W Dart/Kotlin.

Aktualizuj stan. React automatycznie rerenderuje komponent. Jak setState(() => ...) w Dart.

Funkcjonalna aktualizacja - używaj gdy nowy stan zależy od poprzedniego! Bezpieczniejsze.

6.2. useEffect - efekty uboczne

useEffect to hook do obsługi efektów ubocznych: pobierania danych z API, subskrypcji, zmian DOM. Odpowiednik initState() + dispose() z Fluttera, ale bardziej elastyczny.

```

import { useEffect } from 'react';

// Przypadek 1: Wykonaj RAZ przy montowaniu komponentu (inicjalizacja)
useEffect(() => {
  loadWeatherData(); // wywołaj async funkcje
}, []); // [] = tablica zależności PUSTA = efekt nie zależy od nic = wykonaj raz
// Odpowiednik: void initState() w Flutter

// Przypadek 2: Wykonaj przy KAZDEJ zmianie city
useEffect(() => {
  if (city) fetchWeather(city);
}, [city]); // [city] = uruchom efekt gdy city się zmieni
// Odpowiednik: didUpdateWidget lub watch() w Compose

// Przypadek 3: Cleanup (sprzątanie po sobie)
useEffect(() => {
  const subscription = locationService.subscribe(onLocation);
  // Funkcja cleanup - wywoływana PRZED następnym efektem lub odmontowaniem:
  return () => subscription.unsubscribe(); // Odpowiednik: dispose() w Dart
}, []);

// Przypadek 4: Pobieranie danych - pełny wzorzec
const [weather, setWeather] = useState<WeatherData | null>(null);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  let cancelled = false; // zapobieganie race condition
  const load = async () => {
    setLoading(true);
    try {
      const data = await weatherService.fetch(city);
      if (!cancelled) setWeather(data);
    } catch (e) {
      if (!cancelled) setError(String(e));
    } finally {
      if (!cancelled) setLoading(false);
    }
  };
  load();
  return () => { cancelled = true; }; // anuluj jeżeli komponent odmontowany
}, [city]);

```

6.3. Context API - globalny stan bez Redux

Gdy wiele komponentów potrzebuje tych samych danych (np. lista ulubionych miast), użyj Context API. To wbudowany w React mechanizm globalnego stanu - odpowiednik Provider z Fluttera lub prostego wzorca ViewModel w Compose.

```

// context/FavoritesContext.tsx - definicja kontekstu
import React, { createContext, useContext, useState } from 'react';

```

```
// Typ kontekstu - co bedziemy udostepniac
interface FavoritesContextType {
  favorites: string[];
  addFavorite: (city: string) => void;
  removeFavorite: (city: string) => void;
}

// Tworzenie kontekstu z wartoscia domyslna (undefined rzuca blad jezeli brak Providera)
const FavoritesContext = createContext<FavoritesContextType | undefined>(undefined);

// Provider - opakowuje dzieci i udostepnia im kontekst
export const FavoritesProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [favorites, setFavorites] = useState<string[]>(['Warszawa', 'Krakow']);

  const addFavorite = (city: string) => {
    if (!favorites.includes(city)) setFavorites(prev => [...prev, city]);
  };

  const removeFavorite = (city: string) => {
    setFavorites(prev => prev.filter(c => c !== city));
  };

  return (
    <FavoritesContext.Provider value={{ favorites, addFavorite, removeFavorite }}>
      {children} /* wszystkie zagniezdzone komponenty maja dostep do kontekstu */
    </FavoritesContext.Provider>
  );
};

// Custom hook - wygodny dostep z walidacja
export const useFavorites = (): FavoritesContextType => {
  const ctx = useContext(FavoritesContext);
  if (!ctx) throw new Error('useFavorites musi byc uzyty w FavoritesProvider!');
  return ctx;
};

// Uzycie w komponencie - bez prop drilling!
const HomeScreen = () => {
  const { favorites, addFavorite } = useFavorites(); // bezposredni dostep
  // ...
};
```

Kiedy Redux? Context vs Redux vs Zustand: Context API wystarczy do prostego stanu globalnego. Dla zlozone aplikacji ze skomplikowanymi zaleznosciami miedzy stanami warto rozwazyc Zustand (prostszy od Redux) lub TanStack Query (do zarzadzania cache danych z API). W tym cwiczeniu Context API jest wystarczajacy.

7. Nawigacja - React Navigation i Expo Router

React Native nie ma wbudowanej nawigacji - musisz dodac biblioteke. Dwa najpopularniejsze rozwiazania to React Navigation (elastyczny, sprawdzony) i Expo Router (oparty na plikach, jak Next.js). W tym cwiczeniu uzyjemy Expo Router, ktory jest domyslny w projektach Expo i bardziej intuicyjny dla nowych deweloperow.

Analogia: system adresow a fizyczna mapa. React Navigation to fizyczna mapa - sam rysujesz drogi i definiujesz polaczenia miedzy miejscami. Expo Router to GPS z adresami URL: sciezka pliku `app/detail/[city].tsx` automatycznie staje sie URL `/detail/Warszawa`. Nie musisz recznie definiowac tras - struktura katalogow jest nawigacja.

7.1. Expo Router - nawigacja oparta na plikach

Plik	Trasa URL	Opis
app/index.tsx	/	Ekran glowny aplikacji.
app/detail/[city].tsx	/detail/Warszawa	[city] = parametr dynamiczny. Odczytaj: const { city } = useLocalSearchParams()
app/(tabs)/_layout.tsx	Konfiguracja zakladek	Nawias = grupowanie bez wpływu na URL. Tabs layout.
app/_layout.tsx	Korzen nawigacji	Stack/Tabs dla calej aplikacji. Zawsze obecny.
app/+not-found.tsx	Nieznana trasa	Ekran 404 dla nieznanych tras.

```
// app/_layout.tsx - korzen nawigacji aplikacji
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen
        name='index'
        options={{ title: 'WeatherApp', headerShown: true }}
      />
      <Stack.Screen
        name='detail/[city]'
        options={({ route }) => ({ title: route.params?.city as string })}
      />
    </Stack>
  );
}

// Nawigacja programatyczna - z dowolnego komponentu:
import { router, useLocalSearchParams } from 'expo-router';

// Przejscie do ekranu szczegolow:
router.push(`/detail/${cityName}`); // dodaj na stos (Back dziala)
router.replace(`/detail/${cityName}`); // zastap biezacy ekran (brak Back)
router.back(); // wroc do poprzedniego ekranu

// Odczyt parametru w [city].tsx:
const { city } = useLocalSearchParams<{ city: string }>();
// city = 'Warszawa' (z URL /detail/Warszawa)
```

7.2. Zakladki - Tab Navigator

```
// app/(tabs)/_layout.tsx - konfiguracja zakladek dolnych
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons'; // paczka ikon Expo

export default function TabLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#0164FF',
        tabBarStyle: { backgroundColor: 'fff' },
      }}
    >
      <Tabs.Screen
        name='index'
        options={{
          title: 'Pogoda',
          tabBarIcon: ({ color }) => <Ionicons name='partly-sunny' size={24} color={color} />,
        }}
      />
    </Tabs>
  );
}
```

```

    />
    <Tabs.Screen
      name='favorites'
      options={{
        title: 'Ulubione',
        tabBarIcon: ({ color }) => <Icons name='heart' size={24} color={color} />,
      }}
    />
  </Tabs>
);
}

```

8. Pobieranie danych z REST API

WeatherApp będzie pobierać dane z Open-Meteo - darmowego, publicznego API pogodowego bez klucza API. Użyję natywnego `fetch()` dostępnego w React Native oraz omówimy popularną alternatywę - `axios`.

8.1. Serwis API z `fetch()` i TypeScript

```

// services/weatherService.ts
const GEO_URL = 'https://geocoding-api.open-meteo.com/v1/search';
const WEATHER_URL = 'https://api.open-meteo.com/v1/forecast';

// Typy odpowiedzi API (surowy JSON od API)
interface GeoResponse {
  results?: Array<{ latitude: number; longitude: number; name: string }>;
}

interface WeatherResponse {
  current: {
    temperature_2m: number;
    relative_humidity_2m: number;
    wind_speed_10m: number;
    weather_code: number;
  };
}

// Funkcja pomocnicza - geolokalizacja nazwy miasta
async function getCoords(city: string): Promise<{ lat: number; lon: number }> {
  const params = new URLSearchParams({ // bezpieczne budowanie URL
    name: city, count: '1', language: 'pl',
  });
  const res = await fetch(`${GEO_URL}?${params}`);
  if (!res.ok) throw new Error(`Geolokalizacja: HTTP ${res.status}`);

  const data: GeoResponse = await res.json();
  if (!data.results?.length) throw new Error(`Miasto '${city}' nie znaleziono`);

  const { latitude: lat, longitude: lon } = data.results[0];
  return { lat, lon };
}

// Główna funkcja - pobierz pogodę dla nazwy miasta
export async function fetchWeather(city: string): Promise<WeatherData> {
  const { lat, lon } = await getCoords(city); // krok 1: geolokalizacja

  const params = new URLSearchParams({
    latitude: String(lat),
    longitude: String(lon),
    current: 'temperature_2m,relative_humidity_2m,wind_speed_10m,weather_code',
    timezone: 'Europe/Warsaw',
  });

  const res = await fetch(`${WEATHER_URL}?${params}`);
  if (!res.ok) throw new Error(`Pogoda: HTTP ${res.status}`);
}

```

```
const data: WeatherResponse = await res.json();
const c = data.current;

// Mapuj surowy JSON na nasz model WeatherData
return {
  city,
  temperature: c.temperature_2m,
  humidity: c.relative_humidity_2m,
  windSpeed: c.wind_speed_10m,
  weatherCode: c.weather_code,
  updatedAt: new Date(),
};
}
```

8.2. Custom Hook - useWeather

Zamiast pisać `useEffect+useState` w każdym komponencie z osobną, wydzielamy logikę do Custom Hook. To wzorec ponownego użytku logiki stanowej - odpowiednik `PersonalnaKlasaNotifera` w Riverpodzie.

```
// hooks/useWeather.ts
import { useState, useCallback } from 'react';
import { fetchWeather } from '../services/weatherService';
import { WeatherData } from '../types/weather';

// Typ stanu asynchronicznego - unia discriminowana
type WeatherState =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: WeatherData }
  | { status: 'error'; message: string };

// Custom Hook - zawsze zaczyna się od 'use'!
export function useWeather() {
  const [state, setState] = useState<WeatherState>({ status: 'idle' });

  // useCallback - memoizuje funkcje (nie tworzy nowej przy każdym renderze)
  // Odpowiednik lazy lub remember { } w Compose
  const fetch = useCallback(async (city: string) => {
    setState({ status: 'loading' });
    try {
      const data = await fetchWeather(city);
      setState({ status: 'success', data });
    } catch (e) {
      setState({ status: 'error', message: String(e) });
    }
  }, []); // [] = funkcja nie zależy od zewnętrznych zmiennych

  return { state, fetch };
}

// Użycie w komponencie - czyste i czytelne:
const HomeScreen = () => {
  const { state, fetch } = useWeather();

  if (state.status === 'loading') return <ActivityIndicator />;
  if (state.status === 'error') return <Text>Błąd: {state.message}</Text>;
  if (state.status === 'success') return <WeatherCard weather={state.data} />;
  return <Text>Wpisz miasto, aby sprawdzić pogodę.</Text>;
};
```

fetch vs axios: `fetch()` vs `axios`: Natywny `fetch()` jest wbudowany i wystarczy do prostych przypadków. `Axios` (npm install axios) oferuje: automatyczną serializację JSON, interceptory (np. do dodawania tokenów), ładniejszą obsługę błędów HTTP, timeout. W projektach produkcyjnych częściej używa się `axios`.

9. Stylowanie - StyleSheet i nowoczesne podejścia

React Native używa podzbioru CSS do stylowania, ale z kilkoma istotnymi różnicami. Nie ma kaskadowości (cascade) - style nie dziedziczą się automatycznie między komponentami. Każdy komponent musi być samodzielnie ostylowany.

9.1. StyleSheet.create - standardowe podejście

CSS (web)	React Native StyleSheet	Uwagi
background-color: #fff	backgroundColor: '#fff'	camelCase zamiast kebab-case. Standard RN.
font-size: 16px	fontSize: 16	Brak jednostek. Liczby = dp (density-independent pixels).
margin: 8px 16px	marginVertical: 8, marginHorizontal: 16	Brak shorthand. Osobne props dla każdego kierunku.
display: flex; flex-direction: column	(domyslnie!)	View jest domyslnie flex column. Nie trzeba dodawać.
border-radius: 8px	borderRadius: 8	Identyczne (bez px).
box-shadow: 0 2px 8px rgba(0,0,0,0.1)	elevation: 4 (Android) + shadow* (iOS)	Dwa oddzielne systemy cieni - patrz przykłady poniżej.
position: absolute; top: 0	position: 'absolute', top: 0	Absolutne pozycjonowanie działa tak samo jak w CSS.

9.2. Flexbox w React Native

React Native używa Flexbox jako jedynego systemu układu. Domyslna oś jest pionowa (flexDirection: 'column'), co różni się od CSS gdzie domyslnie jest pozioma. Pamiętaj o tej różnicy!

```
// Układy Flexbox w React Native:
const styles = StyleSheet.create({
  // Kolumna (domyslnie!) - elementy jeden pod drugim
  column: {
    flexDirection: 'column', // DOMYSLNE w RN (w CSS domyslnie to 'row')
    alignItems: 'center', // Wyrównanie w poprzek osi (poziome dla column)
    justifyContent: 'space-between', // Rozmieszczenie wzdłuż osi (pionowe dla column)
  },

  // Rząd - elementy obok siebie
  row: {
    flexDirection: 'row',
    alignItems: 'center', // Wyrównanie pionowe dla row
  },

  // Rozciągnięcie - zajmij dostępne miejsce (jak Expanded w Dart)
  expand: { flex: 1 },

  // Absolutne pozycjonowanie - badge DEV w rogu karty
  devBadge: {
    position: 'absolute',
    top: 8, right: 8,
    backgroundColor: '#FF3B30',
    paddingHorizontal: 6, paddingVertical: 2,
    borderRadius: 4,
  },
});

// Dynamiczne style - styl zależy od propów lub stanu:
const cardStyle = [
```

```

styles.card,
isHighlighted && styles.highlighted, // warunkowo dodaj styl
{ backgroundColor: theme.surface }, // obiekt stylu inline
];
<View style={cardStyle} />

```

9.3. Responsywnosc - Dimensions i useWindowDimensions

```

import { Dimensions, useWindowDimensions } from 'react-native';

// Statyczna (nie aktualizuje sie przy obrocie ekranu):
const { width, height } = Dimensions.get('window');

// Dynamiczna (aktualizuje sie automatycznie) - uzywaj w komponentach:
const MyComponent = () => {
  const { width, height } = useWindowDimensions(); // hook!
  const isLandscape = width > height;

  return (
    <View style={{ width: width * 0.9 }}> { /* 90% szerokosci ekranu */ }
    <WeatherCard style={{ height: isLandscape ? 100 : 200 }} />
  </View>
  );
};

```

10. Projekt WeatherApp - architektura i struktura

Polaczymy wszystkie poznane elementy w spojną aplikacje pogodowa. WeatherApp pozwoli wyszukac miasto, wyswietli aktualna pogode i prognoza na 7 dni, z mozliwoscia zapisania ulubionych miast.

10.1. Pelna struktura katalogow projektu

```

WeatherApp/
├── app/
│   ├── _layout.tsx           ← FavoritesProvider + Stack nawigacji
│   ├── (tabs)/
│   │   ├── _layout.tsx      ← Konfiguracja zakladek (Home + Ulubione)
│   │   ├── index.tsx        ← Glowny ekran: wyszukiwarka + lista
│   │   └── favorites.tsx     ← Ekran ulubionych miast
│   └── detail/
│       └── [city].tsx        ← Ekran szczegolów pogody
├── components/
│   ├── WeatherCard.tsx      ← Karta aktualnej pogody
│   ├── ForecastTile.tsx     ← Wiersz prognozy dziennej
│   ├── SearchBar.tsx        ← Pole wyszukiwania z przyciskiem
│   └── ErrorView.tsx         ← Widok bledu z przyciskiem Retry
├── context/
│   └── FavoritesContext.tsx  ← Context + Provider + useFavorites hook
├── hooks/
│   ├── useWeather.ts        ← Logika pobierania pogody (aktualnej)
│   └── useForecast.ts        ← Logika pobierania prognozy 7 dni
├── services/
│   └── weatherService.ts     ← Wywolania HTTP do Open-Meteo API
├── types/
│   └── weather.ts            ← Interfejsy: WeatherData, ForecastDay
├── constants/
│   └── config.ts             ← API_URL, COLORS, ICONS_MAP (kod→ikona)

```

10.2. app/_layout.tsx - punkt startowy

```

// app/_layout.tsx - korzen calej aplikacji
import { Stack } from 'expo-router';
import { FavoritesProvider } from '../context/FavoritesContext';

// RootLayout musi byc eksportem domyslnym

```



```
export default function RootLayout() {
  return (
    // Provider na SZCZYCIE drzewa - dostępny dla wszystkich ekranów
    <FavoritesProvider>
      <Stack>
        <Stack.Screen
          name='(tabs)' // Cała grupa zakładek jako jeden ekran
          options={{ headerShown: false }} // Zakładki mają własny nagłówek
        />
        <Stack.Screen
          name='detail/[city]'
          options={({ route }) => ({
            title: (route.params as { city: string }).city,
            headerBackTitle: 'Wróć',
          })
        />
      </Stack>
    </FavoritesProvider>
  );
}
```

11. Zadania do wykonania

Zadania prowadzi przez budowę WeatherApp krok po kroku. Każde kolejne buduje na poprzednim, dlatego nie pomijaj kolejności. Każde zadanie zakończ weryfikacją zanim przejdziesz do następnego.

Zadanie 1 - Środowisko i projekt bazowy

1. Zainstaluj Node.js (v18 LTS). Sprawdź: `node --version`. Skonfiguruj Android Studio z SDK API 33+ i utwórz AVD (Pixel 7, API 34).
2. Utwórz projekt: `npx create-expo-app@latest WeatherApp --template blank-typescript`. Uruchom na AVD: `npx expo start → a`.
3. Zapoznaj się ze strukturą projektu. Zmodyfikuj `app/index.tsx`: zmień tekst powitalny na "WeatherApp by [twoje-imie]". Sprawdź Fast Refresh.
4. Dodaj zależności: `npm install axios @expo/vector-icons`. Zainstaluj typy: `npm install -D @types/react-native`.
5. Utwórz plik `types/weather.ts` z interfejsem `WeatherData` (`city`, `temperature`, `humidity`, `windSpeed`, `weatherCode`, `updatedAt`: `Date`).

WERYFIKACJA: Screenshot AVD z tekstem powitalnym. `node --version` i `npx expo --version` bez błędów. Plik `types/weather.ts` z interfejsem.

Zadanie 2 - Serwis API i custom hook

6. Utwórz `services/weatherService.ts` z funkcją `fetchWeather(city: string): Promise<WeatherData>`. Dwuetapowo: geolokalizacja Open-Meteo → dane pogodowe.
7. Przetestuj serwis w `app/index.tsx` tymczasowo: wywołaj `fetchWeather('Warszawa')` w `useEffect` i wyloguj wynik przez `console.log`. Sprawdź logi w terminalu (Metro bundler).
8. Utwórz `hooks/useWeather.ts` z Custom Hookiem zwracającym `{ state, fetch }`. Stan jest unia discriminowana: `idle | loading | success | error`.
9. Dodaj obsługę błędów: rzuć `WeatherError(message)` dla `HTTP != 200` i pustych wyników geolokalizacji.

WERYFIKACJA: Dane pogodowe widoczne w konsoli Metro. Przetestuj z nieistniejącym miastem - aplikacja pokazuje komunikat błędu (nie crashuje).

Zadanie 3 - Komponenty i pełne UI

10. Zaimplementuj FavoritesContext z lista ulubionych miast, metodami addFavorite() i removeFavorite(). Owijaj aplikację FavoritesProvider w app/_layout.tsx.
11. Skonfiguruj Expo Router: zakładki (tabs) z dwoma ekranami - Główny i Ulubione. Skonfiguruj Stack dla ekranu Detail.
12. Zbuduj HomeScreen: SearchBar (TextInput + przycisk), wyświetlanie stanów loading/error/idle/success, FlatList z przyciskami do dodawania ulubionych.
13. Zbuduj WeatherCard: wyświetla miasto, temperature (duży tekst), wilgotność, prędkość wiatru, ikona pogody na podstawie weatherCode (Ionicons).
14. Zbuduj DetailScreen: wyświetla wszystkie dane dla wybranego miasta + prognoza na 7 dni (osobne wywołanie API: &daily=temperature_2m_max,temperature_2m_min). Wyświetl 7 wierszy ForecastTile.

WERYFIKACJA: Demo na AVD - wyszukanie Warszawy, widok szczegółów z prognozą 7 dni, nawigacja Back, dodawanie do ulubionych, ekran Ulubione z listą.

Zadanie 4 - Ulepszenia i funkcje dodatkowe

15. Zaimplementuj persystencję ulubionych miast przez AsyncStorage (npm install @react-native-async-storage/async-storage). Ulubione mają przetrwać restart aplikacji.
16. Dodaj ciemny motyw: użyj useColorScheme() z react-native aby automatycznie przełączać kolory między jasnym a ciemnym systemem. Wszystkie kolory przez obiekt Colors (nie hardkodowane).
17. Zaimplementuj animacje przez Animated API: FlatList z animowanym wejściem kart (FadeIn z opóźnieniem $\text{index} * 80\text{ms}$). Karta WeatherCard zwiększa się przy naciśnięciu (scale 0.97).
18. Dodaj Pull-to-Refresh (refreshing + onRefresh na FlatList) odświeżający wszystkie ulubione miasta.

WERYFIKACJA: Ulubione przetrwają restart. Ciemny motyw przełącza się automatycznie. Animacje wejścia widoczne na liście. Pull-to-Refresh odświeża dane.

12. Kryteria oceniania

Zadanie	Punkty	Co weryfikuje prowadzący
Zad. 1: Środowisko	20 pkt	Działający AVD, projekt skompilowany, typy WeatherData, zależności dodane.
Zad. 2: API i Hook	25 pkt	fetchWeather() zwraca dane. Custom Hook z unią stanów. Obsługa błędów.
Zad. 3: UI	35 pkt	SearchBar, WeatherCard, DetailScreen z prognozą 7 dni, nawigacja, stany UI.
Zad. 4: Polisz	20 pkt	AsyncStorage, ciemny motyw, animacje wejścia, Pull-to-Refresh.
RAZEM	100 pkt	

Ocena	Punkty / Wymagania
5.0	90-100 pkt - Wszystkie zadania. Ciemny motyw, AsyncStorage, animacje. Brak hardkodowanych kolorów.
4.5	80-89 pkt - Zadania 1-3 kompletne + przynajmniej 2 elementy Zadania 4.
4.0	70-79 pkt - Zadania 1-3. Kompletny UI: wyszukiwarka, szczegóły, prognoza 7 dni.
3.5	60-69 pkt - Zadania 1-2 + HomeScreen i DetailScreen (bez prognozy).
3.0	50-59 pkt - Zadania 1-2. API działa, dane wyświetlane.
2.0	0-49 pkt - Projekt nie kompiluje lub API nie zwraca danych.

13. React Native vs Flutter - tabela porownawcza

Jesli znasz Flutter z poprzednich cwiczen, ta tabela pozwoli szybko znalezc odpowiedniki. Różnice sa realne, ale koncepcje sa zaskakujaco podobne.

Flutter / Dart	odpowiednik React Native / TypeScript
<code>Widget</code> (klasa)	<code>Komponent</code> (funkcja z JSX)
<code>StatelessWidget</code>	Komponent funkcyjny bez <code>useState</code>
<code>StatefulWidget + State</code>	Komponent z <code>useState</code> / <code>useReducer</code>
<code>build(BuildContext ctx)</code>	<code>return (...JSX...)</code> z funkcji komponentu
<code>setState(() { _x = y; })</code>	<code>setX(y)</code> - setter z <code>useState</code>
<code>initState()</code>	<code>useEffect(() => { ... }, [])</code> - pusta tablica
<code>dispose()</code>	<code>return () => cleanup()</code> z <code>useEffect</code>
<code>Provider / ChangeNotifier</code>	Context API + <code>useContext()</code>
<code>Riverpod / AsyncNotifier</code>	Zustand / TanStack Query (zewnętrzne)
<code>Column / Row</code>	View z <code>flexDirection: 'column' / 'row'</code>
<code>ListView.builder</code>	FlatList z <code>renderItem</code>
<code>Text(style: TextStyle(...))</code>	<code><Text style={styles.text}></code>
<code>ElevatedButton(onPressed: fn)</code>	<code><Pressable onPress={fn}></code>
<code>Image.network(url)</code>	<code><Image source={{ uri: url }} /></code>
<code>GoRouter / Navigator</code>	Expo Router / React Navigation
<code>context.push('/detail/\${city}')</code>	<code>router.push(`/detail/\${city}`)</code>
<code>pubspec.yaml</code>	<code>package.json</code>
<code>flutter pub get</code>	<code>npm install</code>
<code>flutter run</code>	<code>npx expo start</code>
<code>Hot Reload (r)</code>	Fast Refresh (automatyczny)
<code>SharedPreferences</code>	<code>AsyncStorage (@react-native-async-storage)</code>
<code>AnimationController + Tween</code>	<code>Animated.Value + Animated.timing()</code>
<code>AnimatedContainer</code>	<code>Animated.View</code> ze stylem z <code>Animated.Value</code>
<code>Hero widget</code>	Shared Element Transition (react-navigation-shared-element)
<code>flutter test</code>	<code>npx jest</code> (lub <code>npm test</code>)
<code>EdgeInsets.all(16)</code>	<code>padding: 16 (StyleSheet)</code>
<code>Colors.blue[500]</code>	<code>'#1565C0'</code> lub <code>theme.colors.primary</code>

14. Najczęstsze błędy i ich rozwiązania

Błąd / Objaw	Przyczyna i rozwiązanie
Invariant Violation: Text strings must be rendered within a <Text> component	Proba wyświetlenia tekstu poza <Text>. Często przypadkowy string w JSX: <code>{ isLoading && 'Loading' }</code> - gdy <code>isLoading=false</code> , <code>false</code> jest ignorowane, ale <code>0</code> nie! Zamien na: <code>{ isLoading ? <Text>Loading</Text> : null }</code> .

Network request failed / TypeError: Network request failed	Brak połączenia sieciowego lub błąd CORS. Na Android emulatorze użyj 10.0.2.2 zamiast localhost. Sprawdź czy emulator ma dostęp do internetu: ustawienia emulatora > sieć.
Cannot read property 'X' of undefined / null	Użycie wartości zanim dane zostaną załadowane. Dodaj warunek: if (!weather) return <Loading />. Lub użyj optional chaining: weather?.city.
useContext zwraca undefined	Komponent używający useContext jest poza Providerem. Sprawdź czy FavoritesProvider opakowuje aplikację w _layout.tsx, NIE w konkretnym ekranie.
FlatList - 'VirtualizedList' warning: You have a large list	ScrollView opakowuje FlatList. Nie należy tego robić. Użyj ListHeaderComponent do dodania nagłówka wewnątrz FlatList zamiast owijac FlatList w ScrollView.
useEffect działa w petli (infinite loop)	Tablica zależności zawiera obiekt lub tablice tworzone przy każdym renderze. Obiekty w JS porównywane są przez referencje. Użyj useMemo lub useCallback dla obiektów w zależności, lub wyodrębni wartość prymitywną: [city.id] zamiast [city].
Expo: Unable to resolve module 'X'	Brakujący pakiet. Wykonaj: npm install X. Jeśli pakiet jest zainstalowany, sprawdź czy import jest poprawny (wielkość liter!). Restart Metro bundler: npx expo start --clear.
Keyboard pushes content up (blokuje UI)	Brak KeyboardAvoidingView. Owin formularz: <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>. Na Androidzie można też ustawić android:windowSoftInputMode='adjustResize' w AndroidManifest.xml.
Image nie wyświetla się (biały prostokąt)	Brak wymiarów na Image. W RN Image MUSI mieć zdefiniowany width i height w style - inaczej ma rozmiar 0. Dodaj style={{ width: 200, height: 150 }} lub użyj resizeMode='contain'.
Cannot update a component from inside the function body of a different component	setState wywołany podczas renderowania innego komponentu. Przenieś wywołanie do useEffect lub handlera zdarzenia.

Narzędzia diagnostyczne dla React Native

```
# Fast Refresh - automatyczne odświeżanie zmian JS
# Shake emulatora lub d w terminalu = menu deweloperskie

# React Native Debugger - przeglądarka z DevTools dla RN:
npm install -g react-native-debugger
# Lub użyj Flipper (Meta) - bardziej zaawansowane debugowanie

# Expo DevTools - diagnostyka przez przeglądarkę:
npx expo start # Otwórz http://localhost:8081/_expo/

# Logi na emulatorze Android (Logcat):
adb logcat *:S ReactNative:V ReactNativeJS:V

# TypeScript - sprawdzenie typów bez budowania:
npx tsc --noEmit

# Analiza bundla - co zajmuje miejsce:
npx expo export && npx react-native-bundle-visualizer

# Reset cache Metro bundler (gdy coś dziwnego się dzieje):
npx expo start --clear
```

Instrukcja Laboratoryjna React Native 1 - TypeScript, Komponenty, Nawigacja i REST API

Programowanie Aplikacji Mobilnych | Katedra Informatyki i Automatyki

Następne ćwiczenie: React Native 2 - Zustand, Animacje, AsyncStorage i Testy